

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

AI LEARNING ASSISTANT

An Emotion-Aware Personalized Learning Support Platform

An Internship Project Report

submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science & Engineering

Submitted by

Rongali Pavan Kumar

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

July 2026

Declaration

I, Rongali Pavan Kumar, hereby declare that the project report entitled "AI Learning Assistant — An Emotion-Aware Personalized Learning Support Platform" submitted to the Department of Computer Science & Engineering is an original work done by me and my team. The results embodied in this report have not been submitted to any other University or Institute for the award of any degree.

- **Date:** July 15, 2026
- **Signature of the Candidate:** Rongali Pavan Kumar

Acknowledgement

I would like to express my sincere gratitude to everyone who supported and guided me throughout the course of this project. Their valuable suggestions, constructive feedback, and constant encouragement played an important role in helping me and my team complete this work successfully.

I am also thankful to the institution for providing the necessary infrastructure, resources, and learning environment that made this project possible. Finally, I extend my heartfelt thanks to my teammates, family, and peers for their continuous support and motivation during this work.

Abstract

- **Project Overview:** AI Learning Assistant is an emotion-aware learning support platform that classifies a student's emotional state from free-text descriptions of their study challenges and responds with tailored, empathetic guidance.
- **Problem:** Students working independently, and educators triaging large volumes of student messages, both lack a fast, consistent way to gauge emotional state and respond appropriately — manual reading and guessing is slow and inconsistent.
- **Solution:** A Streamlit application that runs a student's input through a BiLSTM classifier and a fine-tuned BERT classifier in parallel, blends their scores with a rule-based keyword lexicon, reports confidence and mixed-emotion breakdowns, and generates a personalized response using Gemini AI.
- **Technology:** Built using Python, Streamlit, TensorFlow/Keras (BiLSTM), PyTorch/HuggingFace Transformers (BERT), scikit-learn, NLTK, Plotly, and the Gemini API, with models trained on Kaggle GPU (T4 x2) notebooks.
- **Outcome:** A working, end-to-end emotion-detection and guidance application with dual-model comparison, an analytics dashboard, and CSV-based interaction logging, runnable even before model training completes via a rule-based fallback.

Table of Contents

1. Introduction	6
2. Problem Statement.....	6
3. Objectives	6
4. Scope of the Project.....	6
5. Literature Survey.....	7
6. Feasibility Study	7
7. Software Development Methodology	7
8. System Requirements.....	8
9. Technology Stack	8
10. System Architecture.....	9
11. Project Directory Structure	9
12. Dataset Design	9
13. Emotion Detection Pipeline	10
14. Model Training Workflow (Kaggle GPU)	10
15. Module Documentation.....	10
16. Streamlit UI Components.....	11
17. AI Guidance Module (Gemini)	11
18. Rule-Based Keyword Enhancement.....	11
19. Confidence Scoring & Mixed-Emotion Detection	11
20. Analytics Dashboard	12
21. Developer Guide	12
22. Security & Privacy	12
23. Testing.....	12
24. Deployment	13
25. User Manual.....	13
26. Advantages	14
27. Limitations	14
28. Future Enhancements.....	14
29. Conclusion.....	14
30. References	15
31. Appendix — Abbreviations	15

1. Introduction

Independent study is one of the most common — and most emotionally taxing — modes of learning for students in technical programs. When a learner gets stuck on a difficult topic, existing tools (search engines, forums, static FAQs) can surface information, but none of them recognise or respond to how the student is actually feeling in the moment: whether they are confused, frustrated, bored, curious, or confident.

Educators and teaching assistants face a related problem at scale. Reading through dozens of student messages to gauge sentiment and craft an appropriately empathetic, field-specific reply is time-consuming and inconsistent across staff. AI Learning Assistant was built to close this gap on both sides: it automatically classifies the emotional tone of a student's free-text message and pairs that classification with an AI-generated, emotion-aware response.

This report documents the complete design and development process undertaken during the internship, covering the problem definition, system architecture, dataset and model design, module-by-module functionality, testing strategy, and deployment steps.

2. Problem Statement

Students working independently often need quick, empathetic guidance when they get stuck, but manually searching forums or drafting help requests takes time and does not capture how they are feeling. Likewise, instructors and teaching assistants who triage student queries must repeatedly interpret emotional tone by hand before crafting a response, which slows response cycles and can cause struggling students to be missed.

AI Learning Assistant addresses both sides of this problem directly: an automated NLP pipeline classifies the emotional state behind a piece of text in seconds, surfaces a confidence score and any secondary ('mixed') emotions detected, and generates a supportive, actionable response — removing the manual interpretation bottleneck for educators and giving independent learners an always-available, emotion-aware companion.

3. Objectives

- **Primary Objective:** Build an automated pipeline that classifies a student's free-text input into one of five emotional states — Bored, Confident, Confused, Curious, Frustrated — and generates a matching, supportive response.
- **Secondary Objective:** Provide transparency into model behaviour through dual-model (BiLSTM vs BERT) comparison, per-class confidence scores, and mixed-emotion detection.
- **Technical Objective:** Implement and train a BiLSTM sequence classifier and fine-tune a BERT transformer classifier, and combine both with a rule-based keyword lexicon for a more robust ensemble.
- **AI Objective:** Use Gemini AI to translate the detected emotion and problem description into a short, natural-language, encouraging response with concrete next steps.
- **Educational Objective:** Reduce manual emotional-triage effort for educators and give independent learners faster, more empathetic self-study support.

4. Scope of the Project

- **Current Scope:** Text-based emotion classification across five classes, dual-model (BiLSTM + BERT) inference with rule-based score blending, confidence scoring, mixed-emotion detection, Gemini-generated guidance, a Streamlit UI with model comparison, CSV interaction logging, and a Plotly analytics dashboard.

- **Future Scope:** Multilingual emotion detection, voice-input support, integration with an LMS (Learning Management System), and richer longitudinal analytics per learner (described further in Section 28).
- **Limitations:** Requires internet connectivity for Gemini-generated responses (falls back to templated guidance offline); classification quality depends on the size and diversity of the labelled training dataset, which was intentionally kept small during development for fast iteration.

5. Literature Survey

Before development began, a comparative survey of existing sentiment-analysis tools and generic chatbot/FAQ systems used in educational contexts was carried out to identify functional gaps that AI Learning Assistant could address.

Feature	Generic Sentiment Tools	Standard Chatbots/FAQs	AI Learning Assistant
Emotion Granularity	Positive/Negative/Neutral only	None (keyword matching)	5 learning-specific emotions
Model Transparency	Single black-box model	Rule-based only	Dual-model comparison (BiLSTM vs BERT)
Mixed-State Detection	No	No	Yes — reports secondary emotion
Personalized Guidance	No	Static, generic replies	Yes (Gemini-generated, emotion-aware)
Usage Analytics	No	Rare	Yes — Plotly dashboard over logged interactions

This survey confirmed that while generic sentiment classifiers and educational chatbots are common, an emotion-specific classifier built for the learning context — combined with transparent dual-model comparison and AI-generated, empathy-aware responses — remains uncommon, positioning AI Learning Assistant as a meaningfully differentiated tool.

6. Feasibility Study

- **Technical Feasibility:** Built entirely with open-source Python libraries (TensorFlow, PyTorch, HuggingFace Transformers, Streamlit) and free-tier cloud GPU access via Kaggle Notebooks (T4 x2), making it highly feasible without specialised infrastructure.
- **Economic Feasibility:** No paid infrastructure is required for training; the only ongoing cost is optional Gemini API usage, which offers a generous free tier suitable for a project of this scale.
- **Operational Feasibility:** The Streamlit interface requires no installation beyond running a single command and needs minimal technical familiarity for students or staff to use.
- **Schedule Feasibility:** The iterative development approach described in Section 7 allowed each phase (preprocessing, model training, UI, guidance generation, analytics) to be built and verified independently within the internship timeframe.

7. Software Development Methodology

The project followed an iterative development lifecycle, with each phase's output feeding into and validated by the next phase before moving forward.

1. Environment Setup & Dependency Configuration
2. Dataset Preparation & Preprocessing
3. Model Training on Kaggle GPU (BiLSTM + BERT)
4. Emotion Detection Pipeline Assembly
5. AI-Powered Guidance Integration (Gemini)
6. Streamlit UI Development
7. Analytics & CSV Logging
8. Testing & Optimization

8. System Requirements

Hardware

- **Processor:** Intel i3 or above
- **RAM:** Minimum 4 GB, recommended 8 GB
- **Storage:** Minimum 2 GB free disk space
- **GPU (for training):** Kaggle Notebook GPU T4 x2 (cloud, free tier) — not required locally

Software

- **Operating System:** Windows / Linux / macOS
- **Python Version:** Python 3.8 or above
- **Notebook Environment:** Kaggle Notebooks (GPU-enabled) for model training
- **IDE:** VS Code / PyCharm for application development
- **Web Browser:** Chrome / Edge / Firefox to run and test the Streamlit application

9. Technology Stack

Layer	Technologies Used
Application Framework	Streamlit
Programming Language	Python 3.8+
Sequence Model	BiLSTM (TensorFlow / Keras)
Transformer Model	BERT (bert-base-uncased, fine-tuned via HuggingFace Transformers / PyTorch)
NLP Preprocessing	NLTK (tokenization, stopword removal)
Classical ML Utilities	scikit-learn (train/test split, label encoding)
Generative AI	Gemini API (google-generativeai)
Visualization	Plotly
Data Handling	Pandas, NumPy
Model Training Infrastructure	Kaggle Notebooks — GPU T4 x2
Persistence / Logging	CSV (interaction logs), Pickle (tokenizer/label encoder)

10. System Architecture

The platform follows a single-process pipeline architecture centred on the Streamlit application. User input is preprocessed once and fanned out to two independent classifiers whose outputs are blended with a rule-based lexicon before being summarised into a final verdict; that verdict then drives both the Gemini guidance call and the CSV logger, which in turn feeds the analytics dashboard.

9. Student Browser Client ↔ Streamlit Application Server (local or hosted)
10. Streamlit App → Text Preprocessing (cleaning, stopword removal) via `data_preprocessing.py`
11. Preprocessed Text → BiLSTM Classifier (`emotion_pipeline.py`, TensorFlow/Keras)
12. Preprocessed Text → BERT Classifier (`emotion_pipeline.py`, HuggingFace Transformers)
13. Both Model Outputs → Rule-Based Keyword Blending & Confidence/Mixed-Emotion Scoring
14. Final Emotion Verdict → Gemini Guidance Module (`gemini_guidance.py`) for a personalized response
15. Final Emotion Verdict + Guidance → CSV Logger (`logger.py`) → `logs/interactions.csv`
16. `logs/interactions.csv` → Analytics Dashboard (`analytics.py`, Plotly charts) rendered back in Streamlit

11. Project Directory Structure

```
emotion_learning_assistant/
├── data/
│   └── sample_emotions.csv      # bootstrap dataset
├── models/                     # trained model artifacts
│   ├── bilstm_model.h5
│   ├── tokenizer.pkl
│   ├── label_encoder.pkl
│   └── bert_model/
│       └── bert_label_encoder.pkl
├── logs/
│   └── interactions.csv        # auto-created interaction log
├── notebooks/
│   ├── train_bilstm.py        # Kaggle GPU training script
│   └── train_bert.py          # Kaggle GPU fine-tuning script
├── data_preprocessing.py       # cleaning, tokenization, label encoding
├── emotion_pipeline.py         # core inference:
│   BiLSTM+BERT+rules+confidence+mixed emotion
├── gemini_guidance.py         # Gemini API call + safe fallback templates
├── analytics.py               # Plotly dashboard charts
├── logger.py                  # CSV interaction logging
├── app.py                     # Streamlit UI entry point
└── requirements.txt
```

12. Dataset Design

The classifier is trained on a labelled dataset of short, first-person student statements, each tagged with one of five target emotions: Bored, Confident, Confused, Curious, and Frustrated. During development, a 20-row bootstrap dataset (`data/sample_emotions.csv`) was used to validate the full pipeline end-to-end; this is intended to be replaced with a larger dataset (hundreds to thousands of rows) sourced from Kaggle or curated manually before final model evaluation.

Column	Type	Description
--------	------	-------------

text	String	Raw free-text student statement
emotion	Categorical (5 classes)	Ground-truth label: Bored, Confident, Confused, Curious, Frustrated
clean_no_stop	String (derived)	Cleaned, stopword-removed version used for BiLSTM tokenization

13. Emotion Detection Pipeline

Text preprocessing (`data_preprocessing.py`) lowercases input, strips URLs and punctuation, and removes stopwords while explicitly preserving negation words (e.g. 'not', 'never') since they materially change emotional meaning. The cleaned text is then routed through two independent classifiers.

- **BiLSTM Path:** Tokenized and padded to a fixed length, then passed through an Embedding layer feeding two stacked Bidirectional LSTM layers with dropout, followed by dense layers ending in a 5-way softmax.
- **BERT Path:** The raw (lightly cleaned) text is tokenized with BERT's WordPiece tokenizer and passed through a fine-tuned bert-base-uncased sequence classification head.
- **Rule-Based Blending:** A keyword lexicon per emotion class produces an independent score distribution, which is blended with each model's softmax output (25% rule weight) to reduce the impact of rare misclassifications on short or idiomatic inputs.
- **Confidence & Mixed Emotion:** The top-scoring class becomes the reported emotion with its probability as confidence; any other class within a fixed margin of the top score is also reported as a secondary ('mixed') emotion, capturing cases like Curious + Confused.
- **Graceful Fallback:** If trained model files are not found under `models/`, the pipeline automatically falls back to the rule-based classifier alone, so the application remains fully runnable before Kaggle training completes.

14. Model Training Workflow (Kaggle GPU)

Both models are trained on Kaggle Notebooks with the GPU T4 x2 accelerator enabled. The BiLSTM script (`notebooks/train_bilstm.py`) uses TensorFlow's `MirroredStrategy` to distribute training across both GPUs; the BERT script (`notebooks/train_bert.py`) uses the HuggingFace Trainer API with `fp16` mixed-precision enabled, which automatically utilises both visible GPUs via data parallelism.

- 17.Upload/paste the dataset and preprocessing code into the Kaggle notebook
- 18.Enable GPU T4 x2 and Internet access in notebook Settings
- 19.Run the BiLSTM training cell — saves `bilstm_model.h5`, `tokenizer.pkl`, `label_encoder.pkl`
- 20.Run the BERT fine-tuning cell — saves the `bert_model/` directory and `bert_label_encoder.pkl`
- 21.Zip the `models/` directory and commit the notebook version
- 22.Download the trained artifacts from the notebook's Output panel
- 23.Place the downloaded files into the local project's `models/` folder

15. Module Documentation

Module	Function
<code>data_preprocessing.py</code>	Text cleaning, stopword removal, dataset loading, pickle helpers
<code>emotion_pipeline.py</code>	Loads BiLSTM/BERT models, runs inference, blends rule-based scores, computes confidence and mixed emotions

gemini_guidance.py	Calls the Gemini API for a personalized response; falls back to per-emotion templates if no API key is set
logger.py	Appends every interaction (input, per-model predictions, guidance) to logs/interactions.csv
analytics.py	Builds Plotly charts: emotion distribution, confidence trend, model agreement, mixed-emotion rate
app.py	Streamlit UI — problem input form, model comparison, AI guidance, analytics dashboard

16. Streamlit UI Components

- **Problem Input Form:** A text area for the student's free-text description, a model-choice radio (BiLSTM / BERT / Ensemble), and a toggle to enable Gemini-generated guidance.
- **Model Comparison View:** Side-by-side bar charts of each model's per-class scores when the ensemble mode is selected, so the two models' behaviour can be directly compared.
- **Guidance Panel:** Displays the Gemini-generated (or fallback) response beneath the detected emotion and confidence score.
- **Analytics Tab:** A second Streamlit tab rendering the Plotly dashboard, driven entirely by the CSV interaction log.

17. AI Guidance Module (Gemini)

For every classified interaction, the detected emotion, its confidence, and the original problem text are assembled into a structured prompt sent to the Gemini API (gemini-1.5-flash), instructed to acknowledge the student's emotional state naturally, give one or two concrete next steps, and end with brief encouragement — all in three to five sentences of plain prose.

24. Student submits a problem/feeling description
25. emotion_pipeline.py returns the final emotion, confidence, and mixed-emotion breakdown
26. gemini_guidance.py builds a structured prompt from the emotion, confidence, and text
27. The prompt is sent to the Gemini API (or a template fallback is used if no key is set)
28. The generated guidance text is returned to the Streamlit UI
29. logger.py records the full interaction, including the guidance text, to CSV

18. Rule-Based Keyword Enhancement

Alongside the two deep learning models, a lightweight keyword lexicon maps common phrases (e.g. 'putting me to sleep' → Bored, 'keep getting the same error' → Frustrated, 'want to dig deeper' → Curious) to each of the five emotion classes. This lexicon serves two purposes: it blends into both models' predictions to reduce the impact of noisy or ambiguous phrasing, and it acts as the sole classifier when no trained model is available, keeping the application usable throughout development.

19. Confidence Scoring & Mixed-Emotion Detection

Confidence is reported as the top class's blended probability. To surface emotionally ambiguous inputs — a hallmark of real student communication, where curiosity and confusion frequently co-occur — any class whose score falls within a fixed margin of the top class is also reported as a secondary emotion, giving both students and educators a more nuanced read than a single hard label.

20. Analytics Dashboard

The dashboard aggregates all logged interactions to visualise emotion distribution across all sessions, confidence trends over time, the agreement rate between the BiLSTM and BERT models, and the proportion of single- versus mixed-emotion detections — giving a quick, at-a-glance view of both learner sentiment trends and model reliability.

21. Developer Guide

Adding a New Emotion Class

Extend the EMOTION_CLASSES list in `data_preprocessing.py`, add matching keywords to the KEYWORDS lexicon in `emotion_pipeline.py`, retrain both models on a dataset containing examples of the new class, and add a corresponding guidance template in `gemini_guidance.py`.

Model Loading Strategy

Both models are loaded lazily and cached in module-level dictionaries (`_bilstm_cache`, `_bert_cache`) on first use, avoiding repeated disk reads and speeding up subsequent predictions within the same Streamlit session.

22. Security & Privacy

- **API Key Handling:** The Gemini API key is read from an environment variable (`GEMINI_API_KEY`) and never hard-coded or logged.
- **Data Handling:** Interaction logs are stored locally in a CSV file; no student input is transmitted anywhere except to the Gemini API when guidance generation is enabled.
- **Dependency Hygiene:** All dependencies are pinned in `requirements.txt` to avoid unexpected breaking changes from upstream package updates.
- **Fail-Safe Design:** Every external call (model loading, Gemini API) is wrapped so that failures degrade to a safe fallback rather than crashing the application.

23. Testing

Each module includes a standalone test entry point (`__main__` block) that can be run independently of the Streamlit UI to validate behaviour in isolation.

Test Target	Command	Expected Outcome	Status
Preprocessing	<code>python data_preprocessing.py</code>	Cleans and loads <code>sample_emotions.csv</code> without error	PASS
Emotion Pipeline (fallback mode)	<code>python emotion_pipeline.py</code>	Correct emotion + confidence + mixed-emotion breakdown on 4 hardcoded test sentences	PASS
Gemini Guidance (fallback)	<code>python gemini_guidance.py</code>	Returns a non-empty templated response when no API key is set	PASS
CSV Logging	<code>log_interaction()</code> + <code>read_logs()</code>	New row appended and readable via pandas	PASS
Streamlit App	<code>streamlit run app.py</code>	UI loads, form submits, dashboard renders	PENDING (post model download)

Run Commands

```
python data_preprocessing.py      # sanity-check text cleaning
python emotion_pipeline.py        # run hardcoded prediction tests
python gemini_guidance.py        # test guidance generation
streamlit run app.py              # launch the full application
```

24. Deployment

Step 1 — Local Environment Setup

```
python -m venv venv
venv\Scripts\activate           # Windows
pip install -r requirements.txt
python -c "import nltk; nltk.download('punkt'); nltk.download('stopwords')"
```

Step 2 — Model Training (Kaggle)

Trained model artifacts are produced on Kaggle (Section 14) and downloaded into the local models/ directory before the application is run with real, trained classifiers rather than the rule-based fallback.

Step 3 — Configuration

```
set GEMINI_API_KEY=your-key-here    # Windows
export GEMINI_API_KEY=your-key-here  # macOS/Linux
```

Step 4 — Run

```
python -m streamlit run app.py
```

25. User Manual

1. Launching the Application

30. Open a terminal in the project folder and run `streamlit run app.py`
31. The application opens automatically in the default browser (or visit the printed localhost URL)

2. Describing a Problem

Type a description of what you're working on or stuck on into the text area, choose which model(s) to use, and select Analyze.

3. Reading the Result

The detected emotion and confidence appear immediately, followed by any secondary (mixed) emotions, a side-by-side model comparison chart, and — if enabled — a personalized Gemini-generated response.

4. Viewing the Dashboard

Switch to the Analytics Dashboard tab to see aggregated charts across every interaction logged so far, including emotion distribution and model agreement rate.

26. Advantages

- Detects emotional nuance beyond simple positive/negative sentiment, using labels specific to the learning context.
- Dual-model comparison gives transparency into classifier disagreement rather than a single opaque prediction.
- Mixed-emotion detection captures realistic, ambiguous emotional states.
- Runs end-to-end even before model training completes, via a rule-based fallback.
- Personalized, empathetic guidance generated in real time rather than static canned responses.

27. Limitations

- Classifier accuracy is directly limited by the size of the labelled training dataset used.
- Gemini-generated guidance requires an internet connection and a valid API key; otherwise a simpler templated response is used.
- Currently supports English-language text only.

28. Future Enhancements

- **Multilingual Support:** Extend preprocessing and both classifiers to handle Telugu and other regional languages alongside English.
- **Voice Input:** Accept spoken descriptions of the student's problem via speech-to-text.
- **LMS Integration:** Connect the pipeline to a learning management system to automatically triage incoming student messages.
- **Longitudinal Analytics:** Track an individual learner's emotional trend over a semester to flag sustained frustration or disengagement early.
- **Model Distillation:** Explore a smaller, faster BERT variant (e.g. DistilBERT) to reduce inference latency on CPU-only deployments.

29. Conclusion

AI Learning Assistant successfully combines two complementary deep learning approaches — a BiLSTM sequence model and a fine-tuned BERT transformer — with rule-based keyword enhancement to classify student emotional states, and pairs that classification with Gemini-generated, empathetic guidance. The project also served as a comprehensive practical exercise in NLP pipeline design, deep learning model training on GPU infrastructure, generative AI integration, and interactive dashboard development using Streamlit and Plotly.

30. References

- Ian Goodfellow, Yoshua Bengio, Aaron Courville, Deep Learning, MIT Press, 2016.
- Devlin, J. et al., BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, 2018.
- TensorFlow / Keras Documentation — [tensorflow.org](https://www.tensorflow.org)
- HuggingFace Transformers Documentation — huggingface.co/docs/transformers
- Streamlit Documentation — docs.streamlit.io
- NLTK Documentation — [nltk.org](https://www.nltk.org)
- Gemini API Reference — ai.google.dev/docs

- [Kaggle Notebooks Documentation](https://kaggle.com/docs/notebooks) — kaggle.com/docs/notebooks

31. Appendix — Abbreviations

Abbreviation	Meaning
BiLSTM	Bidirectional Long Short-Term Memory
BERT	Bidirectional Encoder Representations from Transformers
NLP	Natural Language Processing
API	Application Programming Interface
GPU	Graphics Processing Unit
CSV	Comma-Separated Values
UI/UX	User Interface / User Experience
fp16	16-bit Floating Point (mixed precision)
LMS	Learning Management System